

Licenciatura em Engenharia Informática
UC | Matemática Discreta



AquaPoint

João Rua

Evandro Gaspar

Lisboa
10 de dezembro de 2025

INDICE

INDICE..... 2

INTRODUÇÃO 3

O PROBLEMA..... 3

A SOLUÇÃO..... 3

O OBJETIVO..... 4

INTEGRAÇÃO COM MATEMÁTICA DISCRETA 5

INTRODUÇÃO

Aqua Point é uma aplicação mobile cujo permite utilizadores encontrarem bebedouros perto de si, não só para saciarem a sua sede, como também para fornecer a sua opinião sobre o mesmo.

Com recursos intuitivos e práticos, esta aplicação visa simplificar e otimizar a procura de bebedouros funcionais perto do utilizador.

O PROBLEMA

A ideia da aplicação **Aqua Point**, surgiu de um problema, cujo é a falta de bebedouros funcionais perto de cada utilizador.

É muito comum no nosso dia a dia encontrarmos bebedouros que simplesmente não funcionam ou estão em condições muito más.

A criação dessa aplicação visa resolver esses problemas, entregando ao utilizador uma listagem de bebedouros por perto, assim como a apresentação de opiniões e críticas adicionadas por outros utilizadores e ainda a possibilidade de verificar de forma rápida se o bebedouro está funcional ou não.

A SOLUÇÃO

A criação desta aplicação visa resolver os problemas acima listados, entregando ao utilizador uma listagem de bebedouros por perto, a apresentação de opiniões e críticas adicionadas por outros utilizadores e ainda a possibilidade de verificar de forma rápida se o bebedouro está funcional ou não.

O OBJETIVO

O objetivo desta aplicação é sobretudo fornecer ao utilizador um meio de saciar a sua sede e até mesmo evitar que tenha de fazer caminhadas extensas para chegar a um bebedouro que no final de contas, não funciona ou está em péssimas condições, seja na qualidade da água, ou até mesmo a limpeza do mesmo.

Com o **Aqua Point**, é possível verificar todos os bebedouros por perto, adicionar novos em caso de estarem em falta no mapa, adicionar opiniões relativas ao bebedouro sendo elas em prol da higiene ou até mesmo do funcionamento do dispositivo e além disso, conseguir gerir preferências pessoais no perfil da aplicação.

INTEGRAÇÃO COM MATEMÁTICA DISCRETA

```
var finalText = place!!.name
var opacity: Float = 1.0f

if (place?.state_id == 2) {
    finalText = "${place!!.name} ⚠"
    opacity = 0.5f
}
```

O código verifica o estado do bebedouro através da propriedade **state_id**.

Se o valor for **igual a 2**, o sistema indica uma condição de alerta e acrescenta o ícone de alerta ao nome e reduz a opacidade visual do bebedouro.

Permite sinalizar visualmente bebedouros com problemas ou indisponíveis.

```
if (UserDataRepository.getUserId() != null) {
    if (AquaPointsRepository.isFavorite(id)) {
        NetworkService.removeAquaPointFromFavorites(
            userId = UserDataRepository.getUserId(),
            pointId = id
        ) {
            AquaPointsRepository.updateFavoriteAquaPoints(
                userId = UserDataRepository.getUserId()
            ) { pointsData ->
                AquaPointsRepository.setFavoriteCache(pointsData)

                favColor = Color.Gray
            }
        }
    } else {
        NetworkService.addAquaPointToFavorite(
            userId = UserDataRepository.getUserId(),
            pointId = id
        ) {
            AquaPointsRepository.updateFavoriteAquaPoints(
                userId = UserDataRepository.getUserId()
            ) { pointsData ->
                AquaPointsRepository.setFavoriteCache(pointsData)

                favColor = Color.Red
            }
        }
    }
} else {
    navController.navigate(route = Screen.MainPage.route)
}
```

Este código controla o estado de **favoritos**: verifica se o utilizador está com login ativo, adiciona ou remove o bebedouro dos favoritos, atualiza a cache e muda a cor do ícone conforme o estado.

```

repeat( times = 5) { i ->
    val starNumber = i + 1
    Icon(
        imageVector = Icons.Filled.Star,
        contentDescription = "Estrela $starNumber",

        tint = if (starNumber <= rating) StarYellow else Color.Black,

        modifier = Modifier
            .size( size = 30.dp)
            .clickable {
                rating = starNumber
            }
    )
}

```

Este código permite criar cinco **estrelas** clicáveis. Ao selecionar uma, atualiza o `rating` e pinta todas as estrelas até o número de estrelas escolhidos.

```

if (UserDataRepository.getUserId() != null) {
    if (rating > 0 && comment != "" && comment != " ") {
        NetworkService.createNewReview(
            userId = UserDataRepository.getUserId(),
            pointId = currentPlace?.id,
            rating,
            comment
        ) { result ->
            AquaPointsRepository.updateAquaPoints { result ->
                AquaPointsRepository.updateFavoriteAquaPoints( userId = UserDataRepository.getUserId()) { result ->
                    NetworkService.getAquaPointReviews( pointId = currentPlace?.id) { result ->
                        reviews = parseUserReviews( jsonString = result)

                        rating = 0;
                        comment = ""

                        val toast = Toast.makeText(context, text = successMessage, duration = Toast.LENGTH_LONG)
                        toast.show()
                    }
                }
            }
        }
    } else {
        val toast = Toast.makeText(context, text = fillAllFieldsMessage, duration = Toast.LENGTH_LONG)
        toast.show()
    }
}

```

Este código verifica se o **rating** e **comentário** foram preenchidos. Se sim, envia a **review** ao servidor, atualiza os dados locais e a lista de **reviews**, limpa os campos e mostra confirmação. Caso contrário, exibe um aviso de erro.

```

if (aquaPointName != "" && aquaPointName != " " && selectedImageUri != null && selectedOptionLocal != null && selectedOption != null) {
    if (AquaPointsRepository.doesPointExistsByName(aquaPointName)) {
        val toast = Toast.makeText(context, text = point_already_exists, duration = Toast.LENGTH_LONG)
        toast.show()
    } else {
        val inputStream = context.contentResolver.openInputStream( uri = selectedImageUri!!)
        val bytes = inputStream?.readBytes()

        NetworkService.createNewAquaPoint(
            aquaPointName,
            pointType = selectedOption,
            pointLatitude = UserCoordinates.getLatitude(),
            pointLongitude = UserCoordinates.getLongitude(),
            localId = selectedOptionLocal
        ) { responseString ->

```

A imagem acima valida se todos os **campos obrigatórios estão preenchidos** e se o nome do bebedouro não existe na base de dados. Caso exista, mostra uma mensagem de erro. Se estiver tudo correto, carrega a imagem selecionada, prepara os dados inseridos e envia a requisição ao servidor para **criar um bebedouro**.

```

var markerStyle: BitmapDescriptor = BitmapDescriptorFactory.defaultMarker(hue = 200f)
var isFavorite = AquaPointsRepository.isFavorite(id = place.id)

if (isFavorite) {
    if (place.state_id == 2) {
        markerStyle = BitmapDescriptorFactory.fromResource(resourceId = R.drawable.favorite_not_working)
    } else {
        markerStyle = BitmapDescriptorFactory.fromResource(resourceId = R.drawable.favorite_working)
    }
} else if (place.state_id == 2) {
    markerStyle = BitmapDescriptorFactory.defaultMarker(hue = BitmapDescriptorFactory.HUE_ORANGE)
}

```

Acima, no trecho de código, determinamos o **ícone do marcador** no mapa com base no estado do bebedouro e se está marcado como favorito, ajustando o estilo conforme está definido para cada tipo de estado.

```

composable(route = Screen.Add.route) {
    if (UserDataRepository.getUserId() != null) {
        CreateAddAquaPointPage(navController)
        showNavBar.value = true
    } else {
        CreateHomePage(navController)

        showNavBar.value = false
    }
}

```

Este trecho de código controla o acesso à página de adicionar bebedouros. Se o **utilizador** estiver com **login ativo**, abre a página de adição de bebedouros e mantém a barra de navegação visível, caso contrário, redireciona para a página inicial e esconde a barra de navegação.